Increasing Network Security with eBPF

Assignment #1: State of the art

António Lanção (up202205416)
David Sá (up202303580)
Henrique Teixeira (up202306640)

Network Security 2026/27 · Faculty of Sciences, University of Porto

The problem: kernel extension is a trade-off

- LKMs: arbitrary code, full kernel privilege, exact kernel match
- Fixed subsystems: only the policies their authors anticipated
- eBPF: untrusted code, **proven safe before it runs**, attached kernel-wide

> **Thesis:** eBPF is a **substrate**, not a product. Defense systems are built on top, and the substrate is itself attack surface.

Execution and safety model

Lifecycle

- Small fixed instruction set, 11 registers.
- Verifier proves safety statically, before JIT.
- JIT to native code; attach as event handler.
- Maps: the only shared, durable state.

The verifier checks

- Termination: bounded loops, DAG-shaped state.
- Memory safety: range tracking vs. packet bounds.
- Types: per-register type state.
- **Not** intent, **not** authorization: it is the TCB.

Verifier safety ≠ authorization to load. Loading is gated separately: `CAP_BPF`, `CAP_PERFMON`, `unprivileged_bpf_disabled`, BPF tokens.

Hook points: the security decision

Hook	Input	Verdict	Cost
XDP	Raw frame, driver RX	Drop / pass / redirect	Lowest
tc	`sk_buff`, stack	Pass / drop / redirect	Low
cgroup/socket	Workload, socket	Deny / restrict	No skb cost
kprobe/tracepoint	Function args	Observe only	Low
BPF LSM	Security check	Allow / deny	Function entry

Use case A (DDoS): XDP; identification needs header bytes only, and the target of defense is the machine itself.

Use case B (container control): socket/cgroup; identity-aware policy, BPF LSM as the generalization BPFContain takes.

Security systems: same substrate, different authors

System	Kind	Hooks	Overhead
Cilium	Production	XDP / tc / socket	None published
Tetragon	Production	kprobe / tracepoint	None published
bpfbox	Research	LSM + kprobes	≈8.4% web throughput
BPFContain	Research	LSM	Under 16%, on par w/ AppArmor

- Production (Cilium/Tetragon): label identity, orchestration-native policy
- Research (bpfbox/BPFContain): confinement from the threat model up

> **Finding:** what separates them is **policy authorship** and the control plane's exposure , not capability. Overhead is only comparable in the research row.

eBPF itself is attack surface

Adversaries, from the literature

- Over-privileged container: 2.5% of Docker Hub images, no exploit needed
- Verifier/JIT bugs: 22 CVEs in two years
- Policy bypass: TOCTOU, fail-open on event loss
- Shared kernel is NOT a shared boundary: tracing ignores namespaces

> **The uncomfortable finding** ; the the verifier is a hand-written, growing TCB; the literature argues it cannot scale as kfuncs multiply, and cross-container attacks work regardless of what it proved about the programs.

Assignment #2: profiler + rule-based detector

Design

- `perf_event` stack sampling: "what is it doing"
- Tracepoints on exec + syscalls: "what did it become"
- User-space detector: rates, n-grams, z-scores
- Detection only, no enforcement

Evaluation targets

- 100% of threat-model replays detected
- Under 5% CPU at 10 Hz
- FPR reported, not hidden
- Loads with `CAP_BPF` + `CAP_PERFMON` only

> **Security property demonstrated:** detection fidelity **at deployable overhead**, with least privilege (the gap no surveyed system publishes).

Roadmap: deadline 6 December 2026

#	Milestone	Validation	Week
M1	Program loads and attaches	`bpftool prog` shows the hook	1
M2	Controller / telemetry	Events at idle and under load	1
M3	Detector + test cases	Fires on replays, silent on soak	2
M4	Benchmark + baseline	Targets met at 10 Hz	3–4
M5	Privilege audit	`CAP_BPF` + `CAP_PERFMON` only	4
M6	Reproducibility artefacts	Fresh clone reproduces a run	5

35 days from today; 5 working weeks; M6 leaves the final week free for the report and packaging (`TPX_groupY_report_2.pdf`).

Conclusion

- The **hook is the security decision**: same bytecode, three defense regimes
- Production vs. research: policy authorship, not capability
- The verifier is a growing TCB; the kernel is a shared attack surface
- A#2: detection with least privilege, every number published

Questions?